

Lab Task 13

Student Course Manager — Full CRUD with Firebase Realtime Database

Android Studio • Kotlin • Firebase Realtime Database



Instructions

1. Complete the task following all requirements carefully.
2. After completion, push your project to your GitHub repository.
3. UI must be clean, aligned, and properly spaced.
4. All four CRUD operations (Create, Read, Update, Delete) must function correctly.
5. Data must persist in Firebase Realtime Database and be visible in the Firebase Console.



Scenario

You are building a Student Course Manager app for a university. Students can add courses they are enrolled in, view the full list of their courses, update course details, and delete courses they no longer take. All data is stored in Firebase Realtime Database under each student's unique node, making it accessible from any device in real time. The app demonstrates a complete Create-Read-Update-Delete workflow using Firebase.



Project Setup

Project Name	StudentCourseManager
Template	Empty View Activity
Language	Kotlin
Firebase Service	Firebase Realtime Database

Step 1 — Add Firebase Realtime Database

- In Firebase Console → create/reuse your project → Add Realtime Database
- Choose a region and start in test mode (allow all reads/writes for development)
- Add dependencies to build.gradle:

Step 2 — Data Model

Create a data class `Course.kt` to represent each course entry:

✓ Requirements & Screens

Screen 1 — Course List (`MainActivity`)

Layout: `ConstraintLayout` + `RecyclerView`

- Toolbar with app title "My Courses" and a "+" `FloatingActionButton` to add a new course
- `RecyclerView` displaying all courses with a custom `CourseAdapter`
- Each `RecyclerView` item shows: course name, course code, instructor, credit hours, schedule
- Each item has two icon buttons: Edit (pencil icon) and Delete (trash icon)
- Empty state view: show an illustration + "No courses added yet" message when list is empty
- `SearchView` in toolbar to filter courses by name or code (filter locally from the loaded list)
- Real-time listener using `addValueEventListener` so the list updates instantly on any change

Screen 2 — Add Course (`AddCourseActivity`)

Layout: `ConstraintLayout` + `ScrollView`

- Course Name — `EditText` (required)
- Course Code — `EditText`, e.g. CSE301 (required)
- Instructor Name — `EditText` (required)
- Credit Hours — `NumberPicker` or `Spinner`: 1, 2, 3, 4 credits
- Schedule — `EditText`, e.g. "Sun-Tue 10:00 AM"
- Room — `EditText`, e.g. "Room 301, Building B"
- Semester — `Spinner`: Spring 2025, Summer 2025, Fall 2025
- Save Course button — validates all required fields, then pushes to Firebase
- Cancel button — finishes activity without saving
- Show `ProgressBar` while the write operation is in progress

Screen 3 — Edit Course (`EditCourseActivity`)

Layout: Same as `AddCourseActivity` — reuse or extend the layout

- Receives course data via `Intent` (pass the `Course` object as `Serializable`)
- Pre-fills all fields with the existing course data
- Update Course button — validates fields, then calls `setValue()` on the existing node (same ID)
- Show Toast "Course updated successfully" on success
- Delete button available here too — shows confirmation dialog before deleting

Screen 4 — Course Detail (CourseDetailActivity) [optional but recommended]

Layout: NestedScrollView + LinearLayout

- Opens when user taps a course card (not the edit/delete buttons)
- Shows all course fields in a formatted detail view (CardViews or labeled rows)
- Edit FAB at bottom right → navigates to EditCourseActivity